

SOROTI UNIVERSITY
SCHOOL OF ENGINEERING AND TECHNOLOGY
DEPARTMENT OF ELECTRONICS AND COMPUTER ENGINEERING

DESIGN, IMPLEMENTATION AND CALIBRATION OF A MULTI-SENSOR EDGE NODE

WSN2100 — WIRELESS SENSOR NETWORKS

Laboratory 1: Technical Report

Submitted by

DAVIS TWINOMUGISHA
SARAH ASINGWIIRE
OPUCHE SAMSON
PHILLIP OKWANY

in partial fulfilment of the requirements for the degree of

BACHELOR OF ENGINEERING IN ELECTRONICS AND COMPUTER ENGINEERING

SEPTEMBER 2026

TABLE OF CONTENTS

1 OBJECTIVE AND COMPETENCY....1
2 SYSTEM ARCHITECTURE AND HARDWARE....1
3 SENSOR INTERFACES AND SAMPLING DESIGN....2
4 SENSOR CHARACTERIZATION AND DATA QUALITY....3
5 EDGE PROCESSING AND LOCAL DECISION LOGIC....3
6 PYTHON DATA ANALYSIS AND VISUALIZATION....5
7 DISCUSSION AND ENGINEERING INTERPRETATION....7
8 LIMITATIONS AND RECOMMENDED IMPROVEMENTS....8
9 CONCLUSION....8
REFERENCES....8
APPENDIX: REPOSITORY LINK AND FILE STRUCTURE....9

1 OBJECTIVE AND COMPETENCY

The objective of this laboratory is to design, implement and validate a low-cost sensing node that acquires measurements from several sensors, rejects invalid values, processes data locally, makes a simple edge decision and streams machine-readable data for analysis in Python, as a foundation for later IoT analytics and machine-learning work. The node targets a smart-building or environmental-monitoring scenario in a resource-constrained setting.

Competencies demonstrated: programming an ESP32-class microcontroller; interfacing three sensing modalities over digital (single-wire, GPIO) and analog (ADC) interfaces; acquiring, timestamping and validating measurements; local filtering and feature calculation; generating a structured data stream; characterizing sensor response and variability in Python; and documenting a reproducible design through a version-controlled repository.

2 SYSTEM ARCHITECTURE AND HARDWARE

The node has five subsystems (Fig. 1). A sensing layer (DHT22 temperature/humidity sensor, photoresistor light module and PIR motion sensor) feeds an ESP32 DevKit V1, which runs the whole processing chain in firmware: 1 Hz sampling, plausibility validation, linear calibration, a moving-average filter with rate of change, and a threshold decision. A local indicator layer (red LED and buzzer) acts on that decision without a host computer, and a serial CSV stream carries every processed sample to a Python analysis layer. The design was built and validated in the Wokwi simulator [3]; the wiring is shown in Fig. 2.

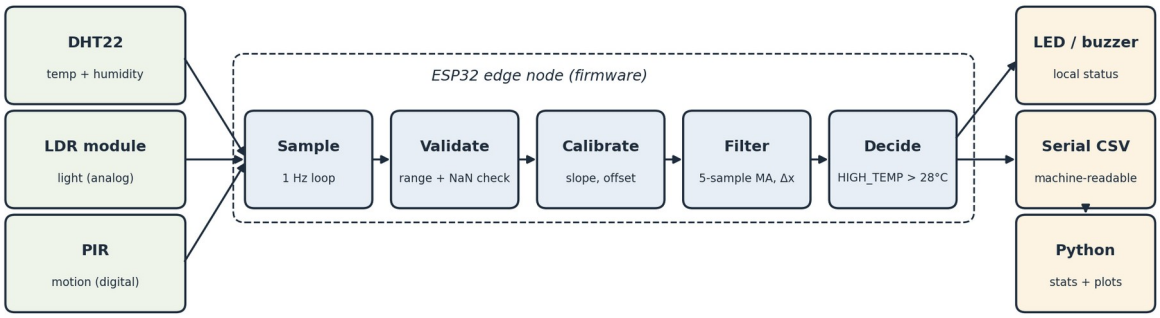


Fig. 1. System architecture, from sensing layer to analysis layer.

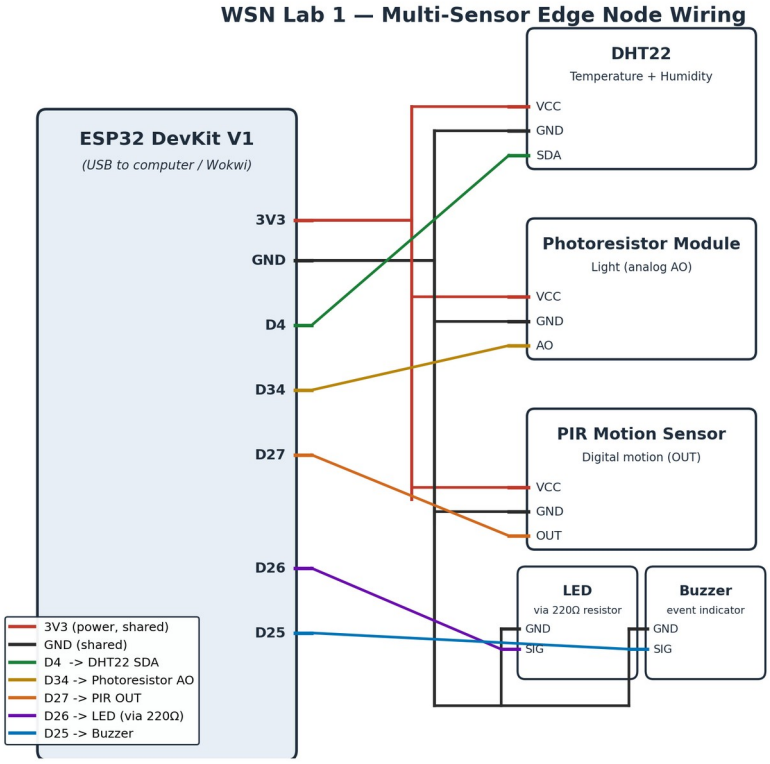


Fig. 2. Circuit/wiring diagram of the simulated sensor node.

Physical prototype: the hardware build uses an Arduino UNO R4 WiFi [8] with a KY-028 NTC thermistor module on A0, a two-pin LDR in a 10 kΩ divider on A1, a PIR sensor on D7, a red LED on D2 and an active buzzer on D8 (firmware in firmware/uno_r4_prototype/). The prototype is fully assembled and operational, running the complete acquisition, filtering, threshold and actuation chain, and is presented in the live demonstration. Its temperature channel was calibrated against a reference thermometer (Section 5). The descriptive statistics, event counts and plots in Sections 4 and 6 are computed from the simulated node's dataset.

3 SENSOR INTERFACES AND SAMPLING DESIGN

Each sensor uses the interface suited to its output (Table 1). All channels are read in a single loop scheduled with millis() at 1 Hz, as the guide recommends. The DHT22, however, specifies a minimum sampling period of 2 s [2], and the Adafruit DHT library returns its cached result when polled faster [4], so the effective rate of fresh temperature and humidity values is 0.5 Hz, while light and motion are genuinely sampled at 1 Hz. The light module is read through ADC1 on GPIO34 (12-bit, 0–4095), which remains usable when Wi-Fi is active.

Sensor / actuator	Interface	ESP32 pin	Supply
DHT22 (temperature, humidity)	Digital, single-wire	GPIO4	3V3 / GND
Photoresistor module (light)	Analog (ADC1)	GPIO34 (AO)	3V3 / GND
PIR motion sensor	Digital input	GPIO27	3V3 / GND
Red LED (HIGH_TEMP indicator)	Digital output	GPIO26 via 220 Ω	GND
Buzzer (motion alert)	Digital output	GPIO25	GND

Table 1. Sensor interfaces and pin assignment.

Every sample is emitted as one comma-separated line with a header row, containing the timestamp, raw temperature, humidity, light, motion, calibrated and filtered temperature, rate of change, status and a validity flag, so it can be parsed directly into a Pandas DataFrame.

4 SENSOR CHARACTERIZATION AND DATA QUALITY

The dataset was captured from the Wokwi simulation over 722 s (723 samples, above the 600-row minimum). Following Section 7.1 of the guide, each sensor was changed deliberately with its on-part control: the temperature slider was stepped through 17 levels between −5.8 °C and 60.8 °C, crossing the 30 °C threshold in use at the time in six separate episodes; the light slider was moved through 22 levels from dark (0.025) to bright (0.989, normalized); and the PIR was triggered in six separate motion episodes. Humidity was not varied in this run. Descriptive statistics are given in Table 2.

Channel	Count	Mean	Min	Max	Std. dev.
Temperature (°C)	723	23.183	−5.800	60.800	12.136
Humidity (%)	723	40.000	40.000	40.000	0.000
Light (normalized 0–1)	723	0.423	0.025	0.989	0.204

Table 2. Descriptive statistics of the simulated capture.

Data quality: no samples failed validation (0 invalid), and the stream had no missing or duplicated samples. The simulated sensors are noise-free: during the undisturbed start of the run every channel had a standard deviation of zero, so the variability in Table 2 comes entirely from the controlled changes rather than from measurement noise, and the simulated sensor has no bias. On the prototype, calibration against the reference thermometer showed that the sensor reads about 0.95 °C low (Section 5). The simulated sensors respond within one sample to each change. The undisturbed baseline lasted 76 s, shorter than the five minutes the guide requires (Section 8). On the physical prototype, the saturated thermistor readings observed during the build were recorded and flagged rather than deleted, as the guide's good-practice rules require.

5 EDGE PROCESSING AND LOCAL DECISION LOGIC

Four operations run on the ESP32 every sample. (i) Validation: a sample is valid only if the DHT22 returns numbers inside its datasheet range (−40 to 80 °C, 0–100 % RH) [2]. (ii)

Calibration: T_cal = a·T_raw + b; on the prototype an offset correction (a = 1, b = +0.95 °C) derived below is applied, while the simulated node uses a = 1, b = 0 because its sensor has no bias. (iii) Filtering and features: a five-sample moving average of T_cal and the rate of change $\Delta x = x(t) - x(t-1)$. (iv) Decision: status = HIGH_TEMP when the filtered temperature exceeds 28.0 °C, otherwise MOTION when the PIR is active, otherwise NORMAL; invalid samples are labelled INVALID.

The decision drives the local indicators directly: the red LED lights while status is HIGH_TEMP and the buzzer sounds while motion is detected. The final firmware adds a 0.5 °C hysteresis band (the alarm clears only below 27.5 °C) so the indicators do not flicker when the temperature hovers at the threshold, and counts each high-temperature or motion episode once, at its start. The dataset in Section 4 was recorded with the previous firmware revision, which applied a 30 °C threshold to the calibrated (unfiltered) temperature, without hysteresis; the event counts in Sections 4 and 6 therefore refer to 30 °C. The IMU feature in item 4 of Section 7.2 was not implemented, as no IMU is fitted.

Calibration of the prototype: the prototype's temperature reading was compared with a reference thermometer placed beside the sensor, recording both values once they had settled. Two paired readings were taken, at about 28 °C and 33 °C (Table 3). The sensor read low at both points, by 1.2 °C and 0.7 °C. An offset model T_cal = T_raw + b was fitted by least squares, giving b = +0.95 °C, which reduced the error against the thermometer from an MAE of 0.95 °C to 0.25 °C (−74 %) and an RMSE of 0.98 °C to 0.25 °C (−75 %) (Fig. 3). A full linear model through the two points (a = 0.881, b = +4.55 °C) fits both exactly, so its residual error is zero by construction and cannot be used to judge accuracy; the offset model was therefore adopted and applied in the prototype firmware as CAL_OFFSET = 0.95 °C. The data and fit are in data/calibration/ and python/calibration.ipynb.

Point	Prototype sensor (°C)	Thermometer (°C)	Error before (°C)
1	28.1	29.3	+1.2
2	32.3	33.0	+0.7

Table 3. Calibration pairs from the prototype and the reference thermometer.

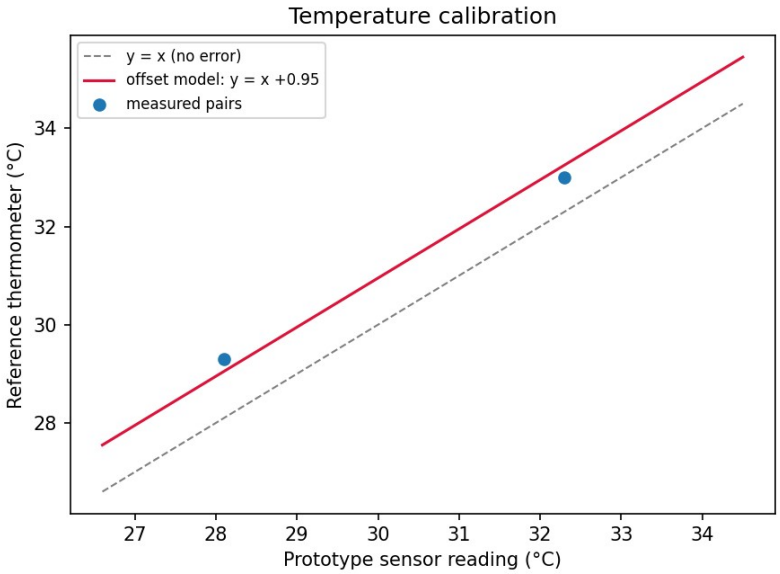


Fig. 3. Temperature calibration: measured pairs and the fitted offset model.

6 PYTHON DATA ANALYSIS AND VISUALIZATION

acquisition.ipynb reads the serial stream with pyserial on hardware, or parses a copied Wokwi Serial Monitor log into CSV. analysis.ipynb loads the CSV into a Pandas DataFrame [5], coerces invalid entries to NaN, computes the statistics required by Section 7.3 (count and duration, mean, minimum, maximum, standard deviation, invalid count and event count) and produces the plots in Figs. 4–8 with Matplotlib [6]. The processed dataset keeps invalid rows with their flag instead of dropping them. A Google Colab version of the notebook is also provided.

Results: 723 observations over 722 s (about 12 minutes); 0 invalid measurements; 90 event samples, made up of 60 HIGH_TEMP samples in 6 episodes and 30 motion samples in 6 episodes.

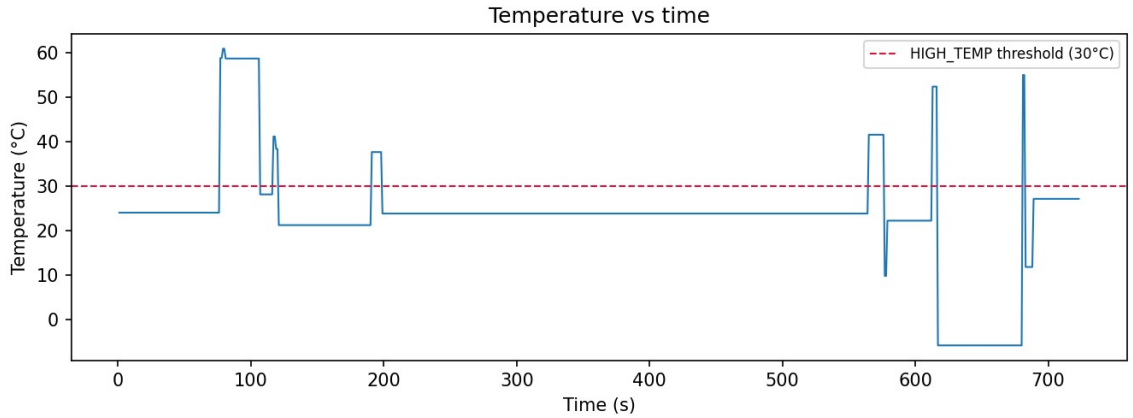


Fig. 4. Temperature vs. time, with the 30 °C threshold used when this dataset was recorded.

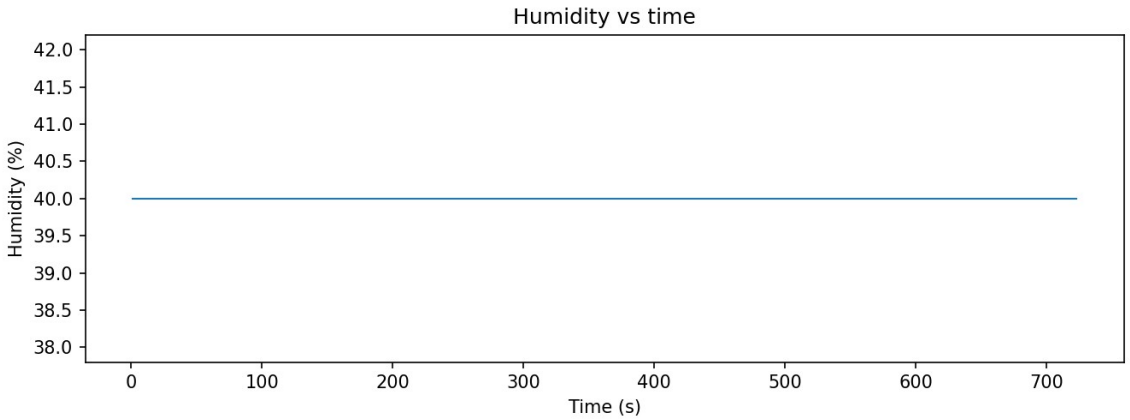


Fig. 5. Humidity vs. time (not varied in this run).

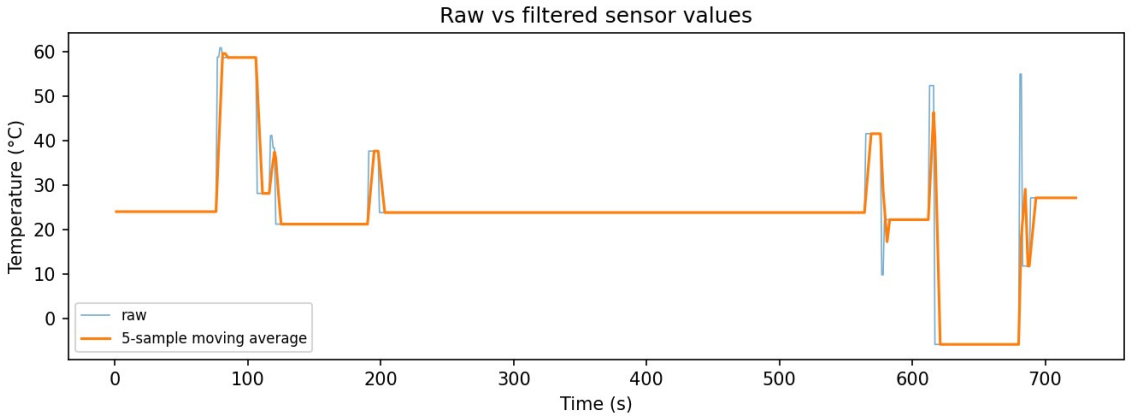


Fig. 6. Raw vs. filtered (five-sample moving average) temperature.

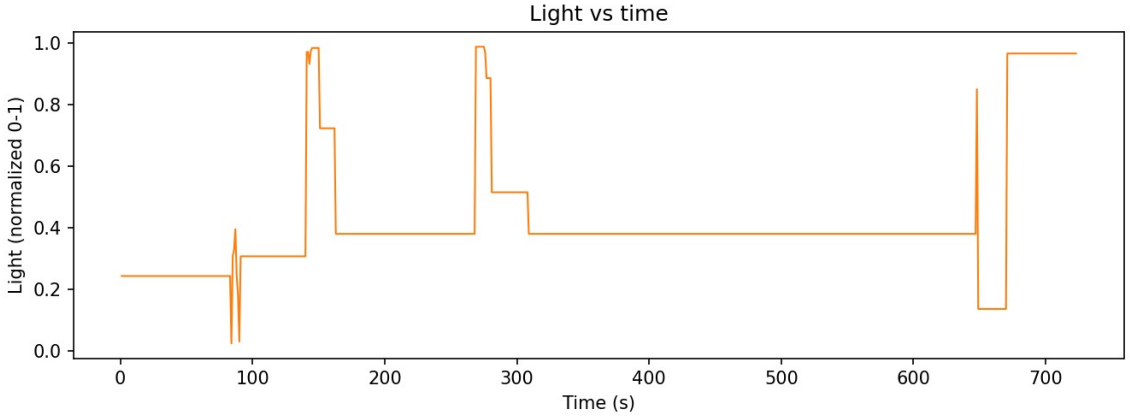


Fig. 7. Light level vs. time during the dark-to-bright sweeps.

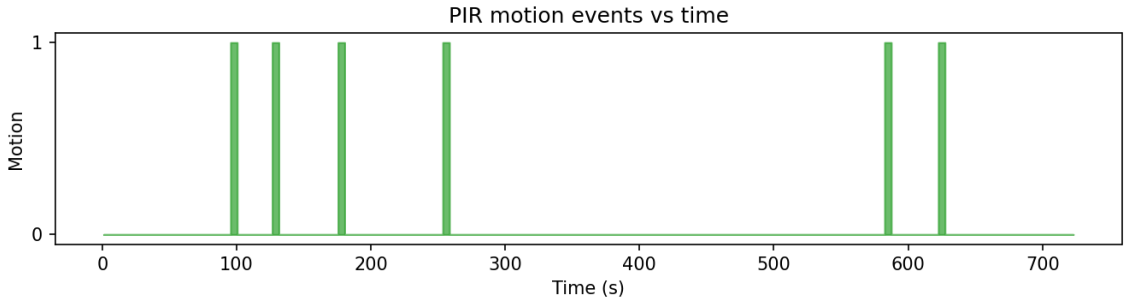


Fig. 8. PIR motion events vs. time.

7 DISCUSSION AND ENGINEERING INTERPRETATION

Q1. Major subsystems and their roles. Sensing (DHT22, LDR module, PIR) converts the environment into signals; the ESP32 acquires, validates, calibrates, filters and decides; the LED and buzzer give the local status; the serial link carries the processed stream; and Python performs the offline analysis.

Q2. Effective sampling rate. Light and motion: 1 Hz, confirmed by 723 samples in 722 s with no gaps. Temperature and humidity: 1 Hz nominal but 0.5 Hz effective, because the DHT22's 2 s minimum period makes every second reading a cached value [2], [4].

Q3. Greatest variability. Temperature (standard deviation 12.1 °C), because it was driven through the widest controlled range to exercise the threshold logic. Humidity showed none, because it was not varied. In simulation this reflects the test protocol, not sensor noise.

Q4–Q5. Systematic error revealed by calibration; change in MAE and RMSE. Calibration against the reference thermometer revealed a systematic negative bias: the prototype's temperature channel reads about 0.95 °C below the true temperature (1.2 °C at 28 °C and 0.7 °C at 33 °C). Applying the offset correction reduced the MAE from 0.95 °C to 0.25 °C (a 74 % reduction) and the RMSE from 0.98 °C to 0.25 °C (75 %). The shrinking error at the higher point suggests a small gain error as well, which more calibration points would quantify.

Q6. Effect of filtering. The five-sample moving average smooths step changes but lags them: after each temperature step the filtered value took four samples (4 s) to come within 10 % of the new level (Fig. 6). The filter trades responsiveness for noise rejection; its noise benefit could not be measured here because the simulated signal has no noise.

Q7. Edge vs. transmitted information. Validation, calibration, filtering and the threshold decision are cheap per-sample operations that also remove bad data at the source, so they belong at the edge. Only validated, calibrated summaries and events need to be transmitted, not every raw sample.

Q8. Energy if every raw sample were transmitted. Each CSV row is about 50 bytes, so a 1 Hz stream is roughly 4.3 MB per day per node (0.9 MB at a 5 s interval). Radio transmission draws hundreds of milliamps on the ESP32, against tens of milliamps for computation [1], so

streaming every raw sample keeps the radio busy and multiplies energy use. Sending summaries or events only is far cheaper.

8 LIMITATIONS AND RECOMMENDED IMPROVEMENTS

Simulated dataset: the descriptive statistics and plots come from the Wokwi simulation, whose sensors are noise-free. Recommendation: log a long-term dataset from the prototype for the same analysis.

Prototype sensing: the KY-028 has no humidity element, so the prototype's humidity field is estimated from temperature; adding a DHT22 would provide measured humidity. Replacing the prototype's empirical temperature scaling with the thermistor Beta equation before calibration would improve accuracy.

Baseline and humidity: the undisturbed baseline was 76 s rather than five minutes, and humidity was not varied. Recommendation: recapture with the final firmware, with at least five minutes of baseline and a humidity sweep.

Calibration range: only two calibration points were taken, over 28–33 °C, so the gain (slope) cannot be determined reliably and the offset correction is valid mainly within that range. Recommendation: add at least three more points across a wider range and refit the linear model.

Sampling vs. DHT22 limit: polling at 1 Hz produces cached temperature and humidity values. Recommendation: sample the DHT22 at 0.5 Hz or flag repeated values.

Scope: the IMU feature and the advanced challenge (local buffering, MQTT, sampling-interval study) were not attempted.

9 CONCLUSION

The simulated node meets the laboratory's functional requirements: three sensing modalities on digital and analog interfaces, plausibility validation, a calibrated temperature channel on the prototype (bias of 0.95 °C corrected, MAE reduced by 74 %), a moving-average filter and rate of change, a 28 °C threshold decision driving an LED and a buzzer, and a machine-readable stream analysed in Python with the required statistics and five plots. The prototype is fully assembled and operational and is shown in the live demonstration. Future work is to log a long-term dataset from the prototype, extend the calibration to more points over a wider range, and add a DHT22 for measured humidity.

REFERENCES

[1] Espressif Systems, "ESP32 Series Datasheet," Espressif Systems. [Online]. Available: https://www.espressif.com/sites/default/files/documentation/esp32_datasheet_en.pdf

[2] Aosong Electronics, "Digital-output relative humidity & temperature sensor/module DHT22 (AM2302)." [Online]. Available: https://www.sparkfun.com/datasheets/Sensors/Temperature/DHT22.pdf

[3] Wokwi, “Wokwi Docs: ESP32 simulation and parts reference.” [Online]. Available: https://docs.wokwi.com

[4] Adafruit Industries, “DHT sensor library,” GitHub repository. [Online]. Available: https://github.com/adafruit/DHT-sensor-library

[5] W. McKinney, “Data structures for statistical computing in Python,” in Proc. 9th Python in Science Conf. (SciPy 2010), 2010, pp. 56–61.

[6] J. D. Hunter, “Matplotlib: A 2D graphics environment,” Computing in Science & Engineering, vol. 9, no. 3, pp. 90–95, 2007.

[7] Soroti University, School of Engineering and Technology, “Wireless Sensor Networks, Laboratory Guide 1: Design, Implementation and Calibration of a Multi-Sensor Edge Node,” 2026.

[8] Arduino, “UNO R4 WiFi,” Arduino Documentation. [Online]. Available: https://docs.arduino.cc/hardware/uno-r4-wifi

APPENDIX: REPOSITORY LINK AND FILE STRUCTURE

Repository: https://github.com/davis-twinomugisha/wsn_lab1

Wokwi simulation: https://wokwi.com/projects/476121306266761217

```
lab1/
├── firmware/sensor_node/   sensor_node.ino, diagram.json (Wokwi)
├── firmware/uno_r4_prototype/  prototype_node.ino (UNO R4 WiFi)
├── python/                 acquisition, analysis, calibration notebooks
├── data/raw/               wokwi_serial_log.csv (+ original dump)
├── data/processed/         cleaned_log.csv
├── data/calibration/       calibration_pairs.csv (prototype vs thermometer)
├── figures/                five analysis plots
├── docs/                   circuit_diagram.png, lab_report.pdf
└── README.md
```